AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_vhdl2_uppgift_2.pdf Version : V1.0	

TEIS ECS - Embedded Computer System -

Darwin Rengifo

2019-01-17

Sammanfattning:
Det är en VGA-kompronent som ska konstruerar med en toppnivå file som innehåller ”metastability_protection” för att synkroniserar asynkron signaler med klockan och säkerställa tillförlitlighet av systemet, ”test_statemachine” för att skickar två test case till dubbelportsminnet och ”vga_component”. Komponenten ”vga_component” innehåller komponenterna ”pll” för att delar 50mhz klockan till 25 mhz på grafikminnet och synkpulser, ”dp_ram(grafikminnet)” och ”vga_sync(synkpulser)”.

Innehåll

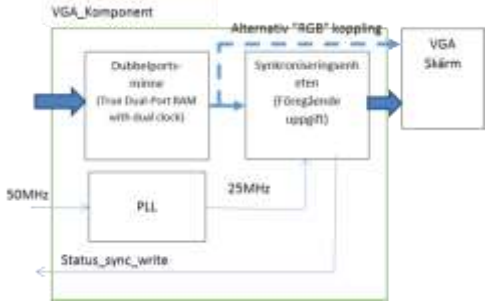
1. INLEDNING 2
2. KRAVSPECIFIKATION 3
3. BESKRIVNINGEN VGA INTERFACET PÅ KORTET 5
4. TESTPROTOKOLL FÖR VERIFIERING OCH ETT ENKELT PROTOKOLL FÖR VALIDERING 6
4.1 TESTPROTOKOLL FÖR VERIFIERING 6
4.2 ENKELT PROTOKOLL FÖR VALIDERING 7
5. KONSTRUKTIONSBESKRIVNING 7
5.1 SYSTEM ARKITEKTUR 7
5.2 DELSYSTEM 8
5.2.1 TOPPNIVÅN 8
5.2.1.1 SYSTEM OCH ARKITEKTUR 8
5.2.2 METASTABILITY - KOMPONENT 9
5.2.2.1 SYSTEM OCH ARKITEKTUR 9
5.2.3 TESTFALLEN - KOMPONENT 10
5.2.3.1 SYSTEM OCH ARKITEKTUR 10
5.2.4 VGA - KOMPONENT 11
6. VERIFIERING 11
6.1 TESTBÄNKEN 11
6.2 BILDER PÅ RESULTATEN FRÅN SIMULERINGEN 11
6.3 IFYLLT TESTPROTOKOLL FÖR VERIFIERING 13
7. VALIDERING 14
8. BILAGA 14

1. INLEDNING

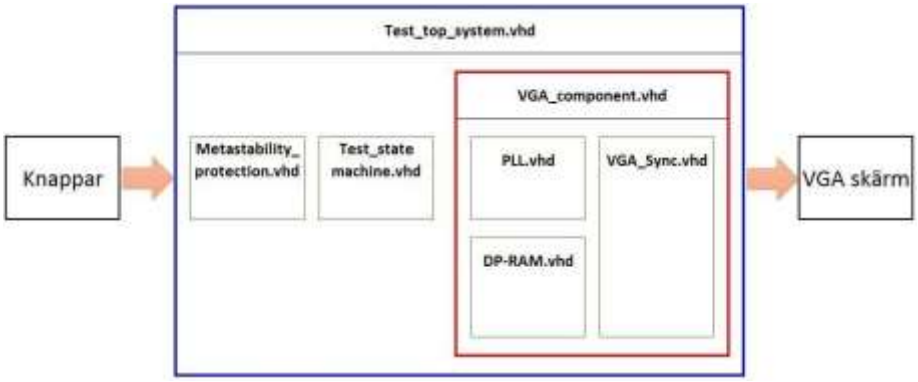
Denna rapport beskriver som bygger en VGA-komponent som styrs av två knappar(aktiveras via tryckknapparna) som genererar två olika testmönster vilka skriver in data i grafikminnet, vga-komponent kommer att läsa grafikminnet och skickar dessa datan mot skärmen. Skärmen och grafikminne har en upplösningen 320*240 pixlar med 3 bitars färg, 7 olika färgkombinationer. Systemet har analyserats genom simulering och verifiering i ModelSim och därefter validerats på ett lämpligt FPGA-kort.

2. KRAVSPECIFIKATION

Tabell 1. Kravspecifikation från vhdl2_uppgift_2

Krav id	Beskrivning	Utfört Ja/nej
	Konstruktionskrav	
1	Skapa ett Quartus-projekt med namnet vhdl2_uppgift_2	
2a	VGA-komponenten (se figur 2), ska kompletteras med dubbelports RAMminnes interface, som är kopplat till en port på dubbelportsminnet som grafikminne. Synkroniseringsenheten (från föregående uppgift) är kopplad till en av portarna på minnet. Använd ett dubbelportsminne med två olika klockor. Det finns VHDL-mallar (True Dual-Port RAM with dual clock) i Quartus. Status_sync_write är aktiv när synkenheten inte läser från dubbelportsminnet. Den finns med för eventuella behov.	
	 **Figur 2: VGA-komponenten** Dubbelportens storlek blir 230 400 bitar = 320*240*3 (grafik minne). Det betyder 320*240 pixlar med 8 färger.	

2b	**Entity krav för VGA komponenten**
	Entity-kraven är att följande in- och utgångar finns med. Andra namn kan användas på topp nivån, t.ex. SW och KEY (samma som i DE1-SoC manualen):
	`entity VGA_component is port( reset_n, clock_50 : IN std_logic;`
	`-- reset_n`
	`-- till VGA enheten`
	`VGA_HS, VGA_VS, VGA_CLK : OUT std_logic;`
	`VGA_BLANK_N : OUT std_logic;`
	`VGA_B, VGA_G, VGA_R : OUT std_logic_vector(7 downto 0);`
	`-- Minnes interfacet`
	`adress_vga_w : IN std_logic_vector(? downto 0);`
	`data_vga_w : IN std_logic_vector(2 downto 0);`
	`write_VGA : IN std_logic;`
	`status_sync_write : OUT std_logic`
	`);`
	`end VGA_component;`	

2c	Konstruera en ”Test_top_system” komponent som innehåller ”metastability_protection”, ”Test_statemachine” och ”VGA_component”. Komponenten ”VGA_component” innehåller komponenterna ”PLL”, ”DPRAM” och ”VGA_Sync” (se figur 3). Valideringen är att ha en ”Test_statemachine” komponent före som exekverar två test case (aktiveras via tryckknapparna) och dessa test case skriver något i dubbelportsminnet (grafikminnet) för att validera konstruktionen. ”Test_statemachine” är konstruerad med en tillståndsmaskin som genererar två olika testmönster vilka skriver in data i grafikminnet.  **Figur 3 Komponent för testsystem**	

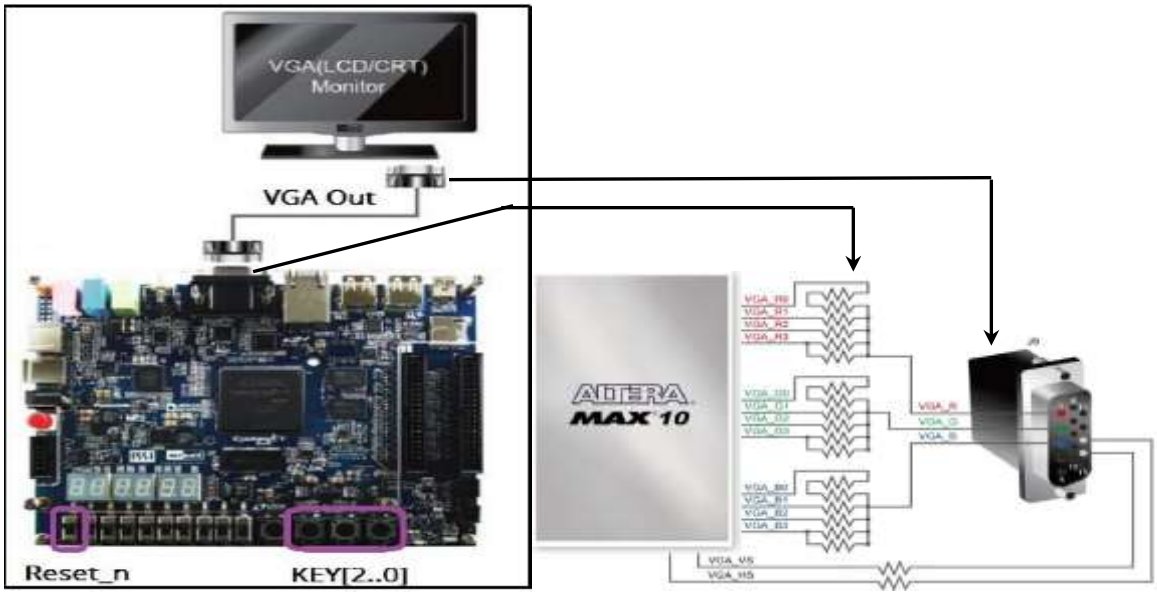
	`entity Test_top_system is port( reset_n, clock_50 : IN std_logic; -- reset_n kopplas till SW9`
	`-- till VGA enheten`
	`VGA_HS, VGA_VS, VGA_CLK : OUT std_logic;`
	`VGA_BLANK_N : OUT std_logic;`
	`VGA_B, VGA_G, VGA_R : OUT std_logic_vector(7 downto 0); -- Start av test case 1 och 2, aktiveras av tryckknapp 1 respektive tryckknapp 2`
	`KEY : IN std_logic_vector(1 downto 0) );`
	`end Test_top_system;`	

2d	VGA-protokollet ska följas och 50MHz klockan ska delas ned med en PLL.	
2e	Metastabilitetssäkra alla relevanta signaler.	
	Testprotokoll	
3	Verifiering: Beskriv test fall. Validering på FPGA kortet: Beskriv två testfall, ett som startas av KEY(0) och det andra med KEY(1). Det ska ritas ut något på skärmen.	
	VHDL-kod	
4	Regler och riktlinjer för VHDL och C ska följas.	
	Verifierings-/Valideringskrav	
5a	Verifiera med ModelSim. Fyll i testprotokollet.	

5b	Validera på DE1-SOC kortet. Fyll i testprotokollet.	
Rapportkrav		
6	Skriv två rapporter: - Uppdatera föregående VGA-rapport med dubbelportsminnet - Kort rapport om testsystem av VGA-komponent: 1) Konstruktionsbeskrivning a. System arkitektur (Test_top_system) b. Delsystem (ner till VGA-komponenten finns beskriven) i. Metastability_protection, ii. Test_statemachine och iii. iii. VGA_component (beskrivning i föregående rapport) 2) Verifiering och validering a. Testprotokollen (någon del t.ex. dubbelports RAM) Visa med ett foto av skärmen att VGA-kontrollern kan få fram något på skärmen (enligt test case 1 och 2).	
Leveranskrav		
7	Leveransen ska ske till Itslearning. Leveransen ska vara två rapporter (VGAkomponenten och Test_top_system) och det arkiverade projektet (Test_top_system). Namnet på filen ska vara ”förnamn_efternamn_vhdl2_uppgift_2”. Sista leveransdag se kursschema. Leveransen ska innehålla rapport och det arkiverade projektet.	

3. BESKRIVNINGEN VGA INTERFACET PÅ KORTET.

DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång. Synkronisering signaler kommer rätt från DE10-Lite kortet och använder en 4-bit DAC som använder en resistens nätverk för att producerar data signaler analoga (röd, grön, blå). På Figur 2 visar som är VGA interfacet på kortet.



Figur 2. Anslutning mellan FPGA kortet DE10-Lite och VGA skärm

4. TESTPROTOKOLL FÖR VERIFIERING OCH ETT ENKELT PROTOKOLL FÖR VALIDERING.

4.1 TESTPROTOKOLL FÖR VERIFIERING

Tabell 2. Testprotokoll för verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Reset_n = '1'	
2	clear_state	All data i minnet är nollställda.	
3	idle_state	Väntar att trycka på button_0 eller button_1	
4	button_0_press_state	Väntar att släpp button_0 och initierar pixel_count med address=3000	
5	button_1_press_state	Väntar att släpp button_1 och initierar pixel_count med address=49000.	
6	pixel_0_button	Skirver värden till grafikminnet från adress 3000 till 69880.	

7	pixel_1_button	Skirver värden till grafikminnet från adress 49000 till 49170.	
8	wait_res_state	Nollställda värden för nästa testfallet	

4.2 ENKELT PROTOKOLL FÖR VALIDERING

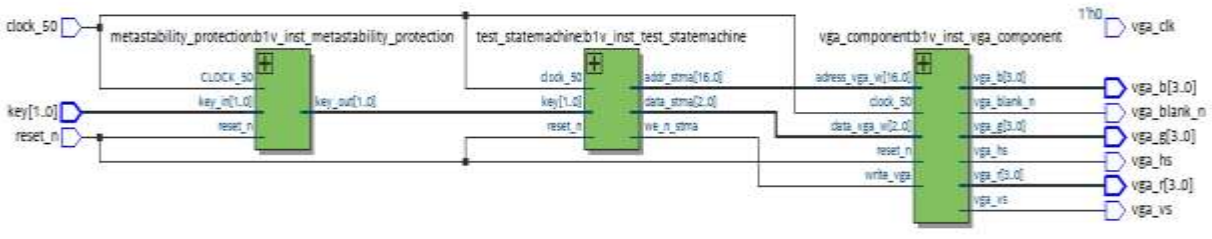
Tabell 3. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	När trycks ned key(0)	Fylla på grafikminnet	
2	När släpp key(0)	Rita en vertikal röd linje	
3	När trycks ned key(1)	Fylla på grafikminnet	
4	När släpp key(1)	Rita en horisontell grön linje	

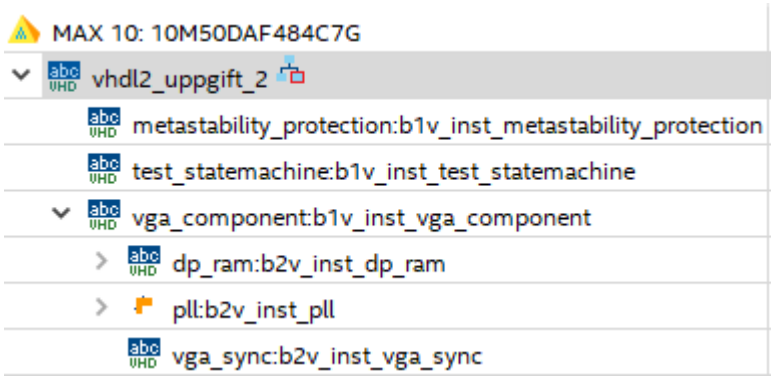
5. KONSTRUKTIONSBESKRIVNING

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm och två knappar för att testar testfallen som skriver i grafikminnet.

5.1 SYSTEM ARKITEKTUR



Figur 3. Systemarkitekturen för TEIS datorsystem



Figur 4. Komponenthierarki för VGA-Kontroller

5.2 DELSYSTEM

Systemet innehålla följande komponenter:

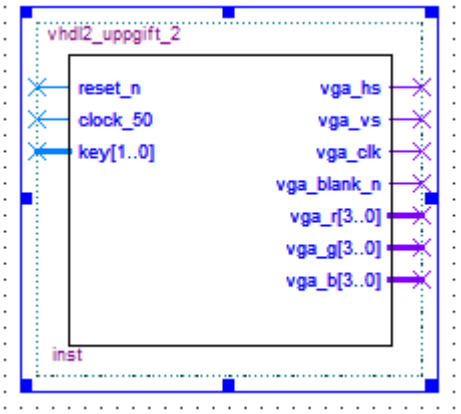
5.2.1 TOPPNIVÅN

Komponent namn: ***vhdl2_uppgift_2***

5.2.1.1 SYSTEM OCH ARKITEKTUR

Den toppnivån användas för att får ingången signaler, instansierar de andra komponenter och skickar utgången signaler till skärmen.

TOPPNIVÅN symbol visas i Figur 5 och tabell 4 visar In- och utsignalerna.



Figur 5. Toppnivån för VGA-kontroller

Tabell 4. In- och utsignalerna till topp filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Test fallet	key	in	std_logic_vector(1 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)

Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
key: For att visa på skärmen två testfallen.

Utgången signaler
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs : Vertikal synkronism. Kontrollerar början och slutet på en ram(frame) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

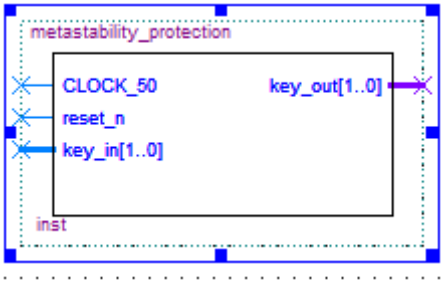
5.2.2 METASTABILITY - KOMPONENT

Komponent namn: ***metastability_protection***
Instansnamn: ***b1v_inst_metastability_protection***

5.2.2.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att bestämmer metastabilitets egenskaperna hos krets för att säkerställa tillförlitlighet och synkroniserar asynkron signaler med klockan.

METASTABILITY-komponentens symbol visas i Figur 6 och tabell 5 visar In- och utsignalerna.



Figur 6. Metastability egenskaperna för VGA-kontroller

Tabell 5. In- och utsignalerna till metastability protection

Signal	Namn	Riktning	Typ
Klocksignal 25 MHz	Clock_50	in	std_logic
Switch reset	reset_n	in	std_logic
Test fallet in	key_in	in	std_logic_vector(1 downto 0)

Test fallet out	key_out	out	std_logic_vector(1 downto 0)

Ingången signaler
clock_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
key_in: Aynkron key.

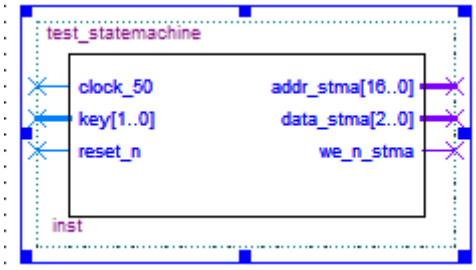
Utgången signaler
key_in: Synkron key.

5.2.3 TESTFALLEN - KOMPONENT

Komponent namn: ***test_statemachine***
Instansnamn**: b1v_inst_ test_statemachine**

5.2.3.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att exekverar två test case (aktiveras via tryckknapparna) och dessa test case skriver något(key_0 en röd linje, key_1 en grön linje) i dubbelportsminnet.
STATEMACHINE-komponentens symbol visas i Figur 7 och tabell 6 visar In- och utsignalerna.



Figur 7. Statemachine för VGA-kontroller

Tabell 6. In- och utsignalerna till statemachine

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	clock_50	in	std_logic
Switch reset	reset_n	in	std_logic
Test fallet in	key_in	in	std_logic_vector(1 downto 0)
Testfall Adress	addr_stma	in	std_logic_vector(16 downto 0)
Testfall Data	data_stma	in	std_logic_vector(2 downto 0)
Styr skrivning	we_n_stma	in	std_logic

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.

key: For att rita fyrkanter i vissa skärmen positioner.

Utgången signaler
addr_stma: skriver adress till grafikminnet.
data_stma: skriver data till grafikminnet.
we_n_stma: aktiverar skrivning på grafikminnet.

5.2.4 VGA - KOMPONENT

Komponent namn: ***vga_component***
Instansnamn: **b1v_inst_ vga_component**

Beskrivning innehåller på följande räpport(darwin_rengifo_vhdl2_uppgift_1a).

6. VERIFIERING

6.1 TESTBÄNKEN

Figur 12 visar teststimuli som bygger på testprotokollet för programmet.

```
clock_50       <= NOT clock_50 after 10 ns;  --50 MHz -- varje 20 ns byta värde
--reset_n_in   <= '0', '1' after 60 ns; -- Först reset_n_in ='0', efter av 50 ns

test_vga : process
begin
 --- Test case 1 -- Reset_n = 0 och brytaren gör inget
   reset_n  <= '0';  -- switchs-reset upp
   key(0) <= '1';    -- Tryckknappen trycks inte
   key(1) <= '1';    -- Tryckknappen trycks inte
   wait for 3*sys_clk_period;

   reset_n  <= '1';  -- switchs-reset upp
   wait for 6*sys_clk_period;

   key(0) <= '0'; --Tryckknappen trycks ner
   wait for 30745*sys_clk_period; -- 1537290ns/20ns = 76864

   key(0) <= '1';          --Tryckknappen släpp
   wait for 30748*sys_clk_period; -- 1537410ns/20ns = 76870

   key(1) <= '0';          --Tryckknappen trycks ner
   wait for 30751*sys_clk_period;

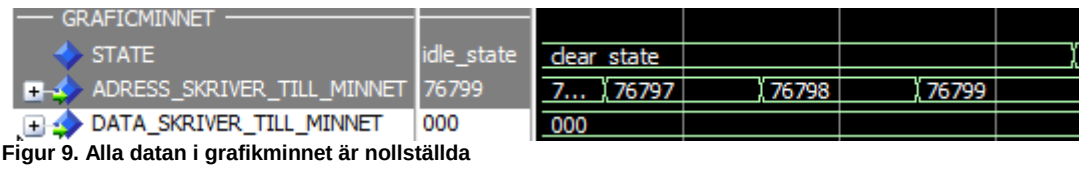
   key(1) <= '1';          --Tryckknappen släpp
   wait for 30754*sys_clk_period; -- 1537410ns/20ns = 76870

wait;                              -- STOPPAR SIMulation
end process test_vga;
```

Figur 8. Tesbänken

6.2 BILDER PÅ RESULTATEN FRÅN SIMULERINGEN

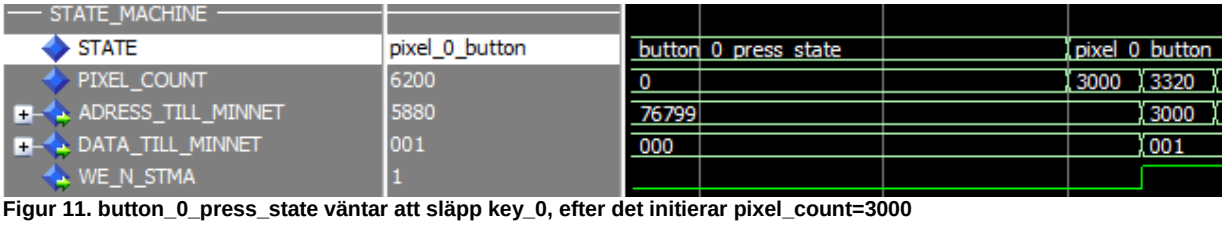
Följande figurer (figur 9-15) visar olika signaler efter verifiering i ModelSim



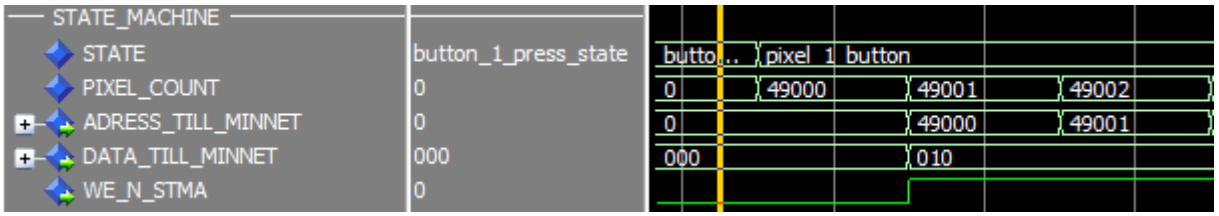
Figur 9. Alla datan i grafikminnet är nollställda



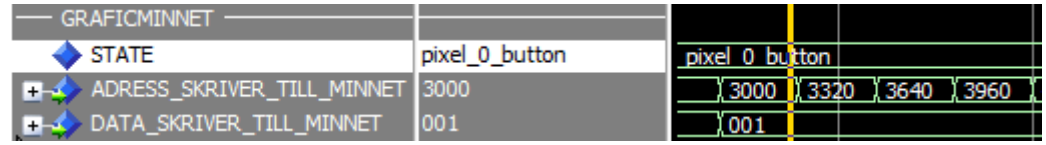
Figur 10. Idle_state väntar att trycker key_0 eller key_1

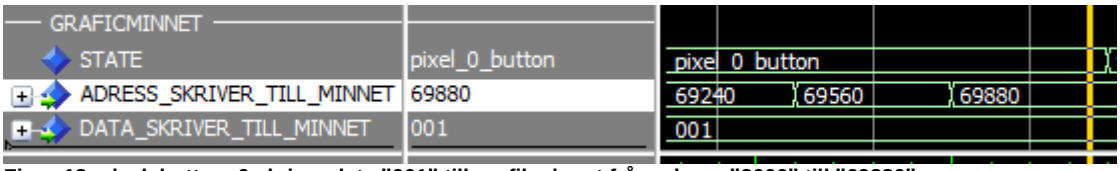


Figur 11. button_0_press_state väntar att släpp key_0, efter det initierar pixel_count=3000

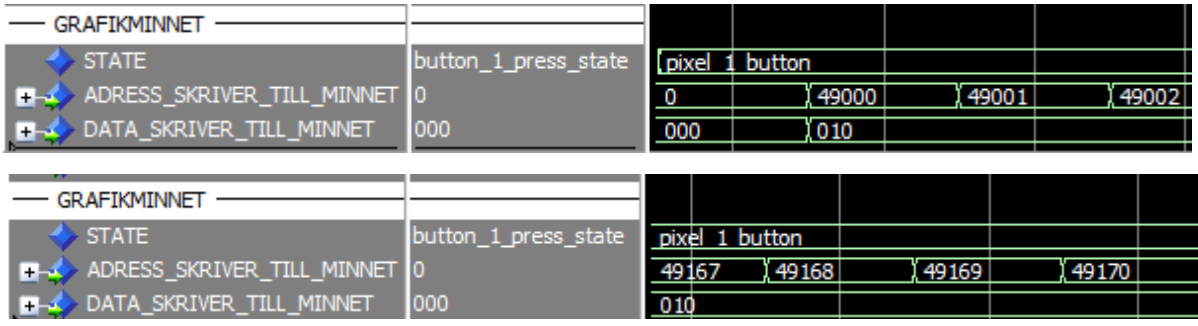


Figur 12. button_1_press_state väntar att släpp key_1, efter det initierar pixel_count=49000

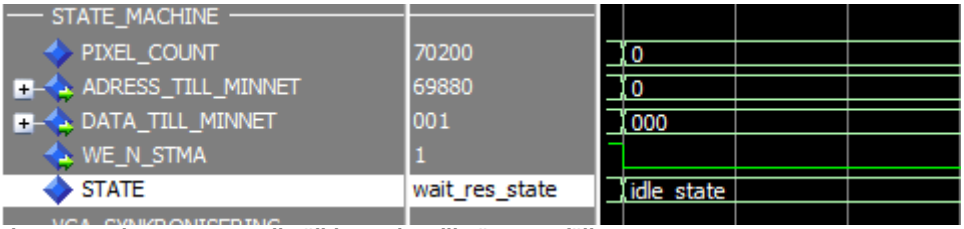




Figur 13. pixel_button_0 skriver data ”001” till grafikminnet från adress ”3000” till ”69880”



Figur 14. pixel_button_1 skriver data ”010” till grafikminnet från adress ”49000” till ”49170”



Figur 15. wait_res_state nollställda varden till nästa testfällen

6.3 IFYLLT TESTPROTOKOLL FÖR VERIFIERING

Tabell 7 visar testprotokoll för verifiering av VGA-kontroller.

Tabell 7. Testprotokoll för verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Reset_n = '1'	Ja
2	clear_state	All data i minnet är nollställda.	Ja
3	idle_state	Väntar att trycka på button_0 eller button_1	Ja
4	button_0_press_state	Väntar att släpp button_0 och initierar pixel_count med address=3000	Ja
5	button_1_press_state	Väntar att släpp button_1 och initierar pixel_count med address=49000.	Ja
6	pixel_0_button	Skirver värden till grafikminnet från adress 3000 till 69880.	Ja
7	pixel_1_button	Skirver värden till grafikminnet från adress 49000 till 49170.	Ja
8	wait_res_state	Nollställda värden för nästa testfallen	Ja

Följande figurer (figur 7-13) visar olika signaler efter verifiering i ModelSim.

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet.

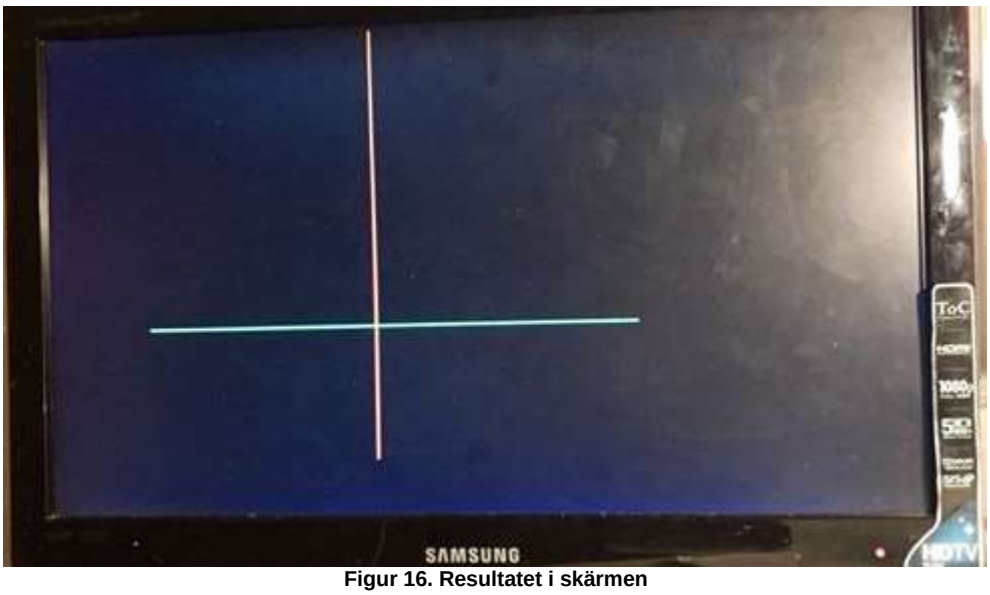
DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm och tre brytarna för att kan testar VGA-kontroller.

7. VALIDERING

Tabell 8 visar testprotokoll för validering och figur 20 visar validering i VGA-skärmen.

Tabell 8. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	När trycks ned key(0)	Fylla på grafikminnet	Ja
2	När släpp key(0)	Rita en vertikal röd linje	Ja
3	När trycks ned key(1)	Fylla på grafikminnet	Ja
4	När släpp key(1)	Rita en horisontell grön linje	Ja



Figur 16. Resultatet i skärmen

8. BILAGA

a. VHDL TOPPNIVÅ

```
------------------
library ieee;
use ieee.std_logic_1164.all;
```

```
entity vhdl2_uppgift_2 is

	port
	(
	reset_n		: in std_logic;
	clock_50	: in std_logic;
	key		: in std_logic_vector(1 downto 0);
	vga_hs		: out std_logic;
	vga_vs		: out std_logic;
	vga_clk		: out std_logic:='0';
	vga_blank_n : out std_logic;
	vga_r		: out std_logic_vector(3 downto 0);
	vga_g		: out std_logic_vector(3 downto 0);
	vga_b		: out std_logic_vector(3 downto 0)
	);
end vhdl2_uppgift_2;

architecture rtl of vhdl2_uppgift_2 is
component metastability_protection
	port (
	CLOCK_50	: in std_logic;
	reset_n		: in std_logic;
	key_in		: in std_logic_vector(1 downto 0);
	key_out		: out std_logic_vector(1 downto 0)
	);
end component;
component test_statemachine
	port
	(
	clock_50	: in std_logic;			-- Clock input
	key		: in std_logic_vector(1 downto 0);-- Button input (key_0, key_1)
	reset_n		: in std_logic;					-- Reset_n
	addr_stma	: out std_logic_vector(16 downto 0); -- sktiver adress till RAM
minne
	data_stma	: out std_logic_vector(2 downto 0); -- sktiver data till RAM minne
	we_n_stma	: out std_logic			-- skriver eller läsa på RAM minne
	);
end component;

component vga_component
	port
	(
```

```
        reset_n                                 : in std_logic;
        clock_50                                : in std_logic;
        adress_vga_w            : in std_logic_vector(16 downto 0);
        data_vga_w              : in std_logic_vector(2 downto 0);
        write_vga               : in std_logic;
        status_sync_write : out std_logic;
        vga_hs                                  : out std_logic;
        vga_vs                                  : out std_logic;
        vga_clk                                 : out std_logic;
        vga_blank_n  : out std_logic;
        vga_r                                   : out std_logic_vector(3 downto 0);
        vga_g                                   : out std_logic_vector(3 downto 0);
        vga_b                                   : out std_logic_vector(3 downto 0)
                    );
        end component;

        signal tmp_addr_stma : std_logic_vector(16 downto 0) := (others => '0');
        signal tmp_data_stma : std_logic_vector(2 downto 0) := (others => '0');
        signal tmp_we_n_stma, tmp_status_sync_write, tmp_vga_clk : std_logic := '0';
        signal tmp_key_out          : std_logic_vector(1 downto 0) := (others => '1');


begin

        b1v_inst_metastability_protection : metastability_protection
        port map(
        CLOCK_50    => clock_50,                -- input clock
        reset_n     => reset_n,                 -- asynkron reset
        key_in      => key,                     -- input key till testfallet
        key_out     => tmp_key_out -- output key till test_statemachine
        );

        b1v_inst_test_statemachine : test_statemachine
        port map(
        clock_50    => clock_50,  -- input clock
        key         => tmp_key_out, -- input key till test_statemachine
        reset_n     => reset_n,    -- asynkron reset
        addr_stma   => tmp_addr_stma,-- output adress till vga_component
        data_stma   => tmp_data_stma,           -- output data till vga_component
        we_n_stma   => tmp_we_n_stma            -- output till vga_component
        );

        b1v_inst_vga_component : vga_component
```

```
        port map(
        reset_n         => reset_n,   -- asynkron reset
        clock_50        => clock_50, - input clock
        adress_vga_w                  => tmp_addr_stma, -- input adress till dp_ram
        data_vga_w                    => tmp_data_stma,          -- input data till dp_ram
        write_vga                     => tmp_we_n_stma, -- input skriver till dp_ram
        status_sync_write => tmp_status_sync_write,
        vga_hs          => vga_hs,    -- output horizontell sync
        vga_vs          => vga_vs,    -- output vertikall sync
        vga_clk         => tmp_vga_clk,
        vga_blank_n  => vga_blank_n, -- sync pulses
        vga_r           => vga_r,     -- display färg röd
        vga_g           => vga_g,     -- display färg grön
        vga_b           => vga_b      -- display färg blå
        );
    end rtl;
```

b. WAVE-FIL

```
    onerror {resume}
    quietly WaveActivateNextPane {} 0
    add wave -noupdate -divider -height 20 TOP_NIVÅ
    add wave -noupdate -label CLOCK_50 /vhdl2_uppgift_2_vhd_tst/clock_50
    add wave -noupdate -label KEY /vhdl2_uppgift_2_vhd_tst/key
    add wave -noupdate -label RESET /vhdl2_uppgift_2_vhd_tst/reset_n
    add wave -noupdate -label RÖD /vhdl2_uppgift_2_vhd_tst/vga_r
    add wave -noupdate -label GRÖN /vhdl2_uppgift_2_vhd_tst/vga_g
    add wave -noupdate -label BLÅ /vhdl2_uppgift_2_vhd_tst/vga_b
    add wave -noupdate -label BLANK /vhdl2_uppgift_2_vhd_tst/vga_blank_n
    add wave -noupdate -label VGA_CLK /vhdl2_uppgift_2_vhd_tst/vga_clk
    add wave -noupdate -label HORISONTELL_SYNC /vhdl2_uppgift_2_vhd_tst/vga_hs
    add wave -noupdate -label VERTIKAL_SYNC /vhdl2_uppgift_2_vhd_tst/vga_VS
    add wave -noupdate -divider -height 20 STATE_MACHINE
    add wave -noupdate -label PIXEL_COUNT
    /vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_test_statemachine/pixel_cnt
    add wave -noupdate -label ADRESS_TILL_MINNET -radix unsigned
    /vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_test_statemachine/addr_stma
    add wave -noupdate -label DATA_TILL_MINNET
    /vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_test_statemachine/data_stma
    add wave -noupdate -label WE_N_STMA
    /vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_test_statemachine/we_n_stma
    add wave -noupdate -label STATE
    /vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_test_statemachine/state
    add wave -noupdate -divider -height 20 VGA_SYNKRONISERING
```

```
add wave -noupdate -label VERTIKAL_COUNTER
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_vga_sync/y_counter
add wave -noupdate -label HORISONTELL_COUNTER
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_vga_sync/x_counter
add wave -noupdate -label ADRESS_UPPLÖSNING -radix unsigned
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_vga_sync/addr_sync
add wave -noupdate -divider -height 20 GRAFIKMINNET
add wave -noupdate -label CLOCK_50_SKRIVER
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/clock_50
add wave -noupdate -label CLOCK_25_LÄSA
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/clk_25
add wave -noupdate -label ADRESS_TILL_VGA -radix unsigned
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/addr_sync
add wave -noupdate -label DATA_TILL_VGA
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/data_vga
add wave -noupdate -label WE_N_STMA
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/we_n_stma
add wave -noupdate -label ADRESS_SKRIVER_TILL_MINNET -radix unsigned
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/addr_stma
add wave -noupdate -label DATA_SKRIVER_TILL_MINNET
/vhdl2_uppgift_2_vhd_tst/i1/b1v_inst_vga_component/b2v_inst_dp_ram/data_stma
TreeUpdate [SetDefaultTree]
WaveRestoreCursors {{Cursor 1} {0 ps} 0}
quietly wave cursor active 0
configure wave -namecolwidth 200
configure wave -valuecolwidth 55
configure wave -justifyvalue left
configure wave -signalnamewidth 0
configure wave -snapdistance 10
configure wave -datasetprefix 0
configure wave -rowmargin 4
configure wave -childrowmargin 2
configure wave -gridoffset 0
configure wave -gridperiod 1
configure wave -griddelta 40
configure wave -timeline 0
configure wave -timelineunits ns
update
WaveRestoreZoom {0 ps} {25012884615 ps}
```